

Eine Änderung. Mehrere Reaktionen.

Das Observer Pattern in C# verstehen

Ein kleines Konsolenprojekt mit klassischem Observer Pattern und C#-Events. Für den Einstieg, Schritt für Schritt.

Die Idee in einem Alltagssatz

Stell dir vor, du abonnierst einen Kanal. Sobald dort etwas Neues erscheint, wirst du benachrichtigt. Du musst nicht ständig selbst nachfragen. Andere Menschen können denselben Kanal abonnieren und ihr Abo wieder beenden.

Genau diese Beziehung bauen wir zwischen Objekten: Ein Objekt liefert Neuigkeiten. Andere Objekte melden sich dafür an. Tritt eine passende Änderung ein, ruft die Quelle die angemeldeten Empfänger auf.

Merksatz

Ein beobachtetes Objekt benachrichtigt seine angemeldeten Beobachter über eine Änderung. Jeder Beobachter entscheidet selbst, wie er darauf reagiert.

Unser Beispiel: die Wetterstation

Die Wetterstation bekommt einen neuen Temperaturwert. Eine Anzeige schreibt den Wert in die Konsole. Ein Hitzealarm gibt ab 30 °C eine Warnung aus. Beide bekommen dieselbe Meldung, haben aber unterschiedliche Aufgaben. Später melden wir die Anzeige wieder ab.

Was dein Lehrer wahrscheinlich meint

Die **klassische Variante** organisiert Anmeldung, Abmeldung und Benachrichtigung selbst, mit Interfaces und einer Liste. Die **Event-Variante** verwendet dafür die C#-Sprachmittel event, += und -=. Es sind zwei Umsetzungen derselben Idee.

Historisch wichtig: Events gehörten schon zu C# 1.0. „Bevor es Events gab“ ist für die Geschichte von C# daher ungenau. Für den Unterricht ist die Unterscheidung „manuell implementiert“ und „mit Events unterstützt“ sinnvoll. Unsere klassische Variante nutzt trotzdem heutige C#-Syntax. [1]

So arbeitest du mit diesem Heft

Lies zuerst Seiten 2-7 und starte die klassische Demo. Lies dann Seiten 8-12 und vergleiche die Event-Demo. Auf Seiten 13-16 findest du Stolperfallen, Übungen mit Lösungen und einen Spickzettel. Du brauchst dafür nur einfache Klassen, Methoden und if-Abfragen; die neuen Begriffe werden hier erklärt.

Welches Problem löst das?

Ohne Observer: Die Station kennt jede konkrete Reaktion

```
// Nur ein Gegenbeispiel, nicht unser Projektcode:
public void TemperaturSetzen(int wert)
{
    anzeige.Anzeigen(wert);
    alarm.Pruefen(wert);
    // Später zusätzlich: protokoll.Speichern(wert) ...
}
```

Hier müsste die Station wissen, welche Anzeige und welcher Alarm existieren und welche Methoden sie besitzen. Ein weiterer Empfänger würde oft eine Änderung an der Station erfordern. Aufgaben, die eigentlich getrennt sind, hängen eng zusammen.

Mit Observer: Ein gemeinsamer Vertrag genügt

In unserer klassischen Station steht nur: „Für jeden angemeldeten Beobachter: Aktualisieren aufrufen.“ Die Station kennt den gemeinsamen Vertrag, aber weder die Anzeigelogik noch die Alarmgrenze. Das nennt man **lose Kopplung**: Die Zusammenarbeit braucht weniger Wissen über die konkrete andere Klasse.

Begriff	Bedeutung im Beispiel
Subject / Observable	Die beobachtete Quelle: Wetterstation.
Observer	Ein Empfänger: TemperaturAnzeige oder HitzeAlarm.
Subscribe / Attach	Anmelden: Ab jetzt sollen neue Meldungen ankommen.
Unsubscribe / Detach	Abmelden: Künftige Meldungen nicht mehr erhalten.
Notify / Update	Die Quelle ruft die vereinbarte Reaktion auf.
Publisher / Subscriber	Übliche Namen für Sender und Abonnent bei Events.

Ein Entwurfsmuster ist eine wiederverwendbare Lösungsidee für ein typisches Strukturproblem im Code. Das Observer Pattern ist also weder ein einzelner C#-Befehl noch eine fertige Klasse, die du nur einschalten musst.

Eins zu vielen

Eine Station kann null, einen oder mehrere Beobachter haben. Die Anzahl wird durch die Anmeldung bestimmt. Ein weiterer Beobachter braucht keinen neuen Spezialaufruf in der Wetterstation.

Die Entkopplung ist nicht vollständig: Klassisch kennt die Station das Beobachter-Interface; bei Events kennt sie den Typ der erlaubten Handler. Irgendein Teil des Programms muss Sender und Empfänger außerdem verbinden. Das übernimmt bei uns Demo.cs.

Projekt öffnen und starten

Im ZIP liegt ein vollständiges Konsolenprojekt. Es verwendet ganzzahlige Temperaturen und feste Messwerte. Du kannst dadurch beide Umsetzungen bei genau demselben Ablauf vergleichen.

In drei Schritten

1. ZIP vollständig entpacken. Installiere bei Bedarf das **.NET 8 SDK**. Das SDK enthält die Werkzeuge zum Übersetzen des Quellcodes; nur eine Runtime genügt dafür nicht.
2. Öffne ein Terminal in dem Ordner, in dem `ObserverLernprojekt.csproj` liegt. In Visual Studio kannst du diese Projektdatei direkt öffnen.
3. Starte eine oder beide Varianten:

```
dotnet run
```

```
dotnet run -- klassisch
```

```
dotnet run -- events
```

Ohne Argument laufen beide Demos nacheinander. Die Argumente nach `--` gehen an unser Programm. In Visual Studio startet `Strg+F5` standardmäßig beide Varianten.

Datei / Ordner	Aufgabe
Program.cs	Liest die Auswahl und startet die passende Demo.
Demo.cs	Erzeugt Objekte, meldet sie an und setzt Messwerte.
Klassisch/Vertraege.cs	Die beiden eigenen Interfaces.
Klassisch/Wetterstation.cs	Beobachterliste und Benachrichtigung.
Klassisch/Beobachter.cs	Anzeige und Alarm mit Interface.
MitEvents/	Station, Ereignisdaten und Empfänger mit Events.
Erwartete-Ausgabe.txt	Vollständige, aus dem Code abgeleitete Ausgabe.

Das Projekt zielt auf `net8.0` und benötigt keine externen NuGet-Pakete. Bei einem neueren SDK müssen die passenden `.NET-8`-Komponenten vorhanden sein; alternativ kannst du das Ziel passend zum SDK ändern, etwa auf `net10.0`. Download: [.NET 8 SDK](#).

Prüfstatus: Quellcode und erwartete Ausgabe wurden statisch auf Konsistenz geprüft. In der Erstellungsumgebung war kein `C#/.NET-SDK` verfügbar; deshalb konnte das Projekt hier nicht kompiliert oder ausgeführt werden. Lokal kannst du zuerst `dotnet build` aufrufen.

Klassisch: die Verträge

Ein **Interface** beschreibt, welche Methoden eine Klasse anbieten muss. Es sagt hier noch nicht, was diese Methoden tun. Das ist wie eine gemeinsame Steckverbindung: Unterschiedliche Geräte können denselben Anschluss verwenden.

```
namespace ObserverLernprojekt.Klassisch;

// Jeder Beobachter verspricht, auf diese Methode zu reagieren.
public interface ITemperaturBeobachter
{
    void Aktualisieren(int temperatur);
}

// Gemeinsamer Vertrag für eine beobachtbare Quelle.
public interface ITemperaturQuelle
{
    void Anmelden(ITemperaturBeobachter beobachter);
    void Abmelden(ITemperaturBeobachter beobachter);
}
```

ITemperaturBeobachter: das Versprechen des Empfängers

Jede Klasse, die dieses Interface implementiert, muss `Aktualisieren(int temperatur)` anbieten. `void` bedeutet: Die Methode liefert kein Ergebnis zurück. `int temperatur` ist der neue ganzzahlige Messwert.

Die Station kann dadurch dieselbe Methode auf ganz unterschiedlichen Objekten aufrufen. Bei einer Anzeige wird etwas angezeigt, bei einem Alarm eine Grenze geprüft. Dass derselbe Aufruf je nach Objekt anders umgesetzt wird, heißt **Polymorphie**.

ITemperaturQuelle: der Vertrag der Quelle

Dieser Vertrag bietet Anmelden und Abmelden an. Ein Empfänger wird als `ITemperaturBeobachter` übergeben. Die Quelle muss dafür keine bestimmte Anzeigeklasse verlangen.

Das Quellen-Interface macht die klassische Rollenverteilung sichtbar. Bei einem sehr kleinen Beispiel könnte man es weglassen und die beiden Methoden nur in `Wetterstation` definieren. Entscheidend bleibt, dass die Station ihre Empfänger über den gemeinsamen Beobachtervertrag anspricht.

Warum heißt die Methode nicht zwingend Update?

`Update`, `Notify`, `Attach` und `Detach` sind typische Lehrbuchnamen. Unsere deutschen Namen `Aktualisieren`, `Benachrichtigen`, `Anmelden` und `Abmelden` erfüllen dieselben Aufgaben. Das Pattern hängt an der Zusammenarbeit, nicht an den Methodennamen.

`namespace` ordnet Klassen einem Namensraum zu. So können im Projekt zwei Klassen `Wetterstation` existieren: eine in `Klassisch`, eine in `MitEvents`. Das `I` am Anfang eines Interface-Namens ist eine verbreitete C#-Namenskonvention.

Klassisch: die Wetterstation

```
namespace ObserverLernprojekt.Klassisch;

public class Wetterstation : ITemperaturQuelle
{
    private readonly List<ITemperaturBeobachter> beobachter = new();

    public int? Temperatur { get; private set; }

    public void Anmelden(ITemperaturBeobachter neuerBeobachter)
    {
        ArgumentNullException.ThrowIfNull(neuerBeobachter);
        // Dieselbe Instanz soll nicht doppelt angemeldet werden.
        if (!beobachter.Contains(neuerBeobachter))
            beobachter.Add(neuerBeobachter);
    }

    public void Abmelden(ITemperaturBeobachter alterBeobachter)
    {
        beobachter.Remove(alterBeobachter);
    }

    public void TemperaturSetzen(int neueTemperatur)
    {
        // Nur Änderungen werden gemeldet, nicht gleiche Folgemessungen.
        if (Temperatur == neueTemperatur)
            return;

        Temperatur = neueTemperatur;
        Console.WriteLine($"[Station] Neu: {Temperatur} °C");
        Benachrichtigen(neueTemperatur);
    }

    private void Benachrichtigen(int neueTemperatur)
    {
        // Momentaufnahme: Abmelden im Callback verändert diese Kopie nicht.
        foreach (ITemperaturBeobachter b in beobachter.ToArray())
            b.Aktualisieren(neueTemperatur);
    }
}
```

Liste: Sie speichert die angemeldeten Empfänger. `readonly` verhindert das Austauschen der Listenreferenz, nicht das Hinzufügen und Entfernen von Einträgen. `Contains` verhindert bei unseren Klassen doppelte Anmeldungen derselben Instanz.

Zustand zuerst: Temperatur wird gespeichert, bevor die Empfänger aufgerufen werden. Der öffentliche Getter erlaubt Lesen; `private set` erlaubt Änderungen nur innerhalb der Klasse.

Startwert: `int?` erlaubt zusätzlich null, also „noch keine Messung“. Deshalb wird auch eine erste Messung von 0 °C weitergegeben. Der Vergleich unterdrückt nur gleiche Folgemessungen.

Benachrichtigen: Die `foreach`-Schleife ruft jede gespeicherte Instanz auf. `ToArray` erstellt eine Momentaufnahme, damit eine Abmeldung während eines Callbacks nicht die gerade durchlaufene Sammlung verändert. Die Grenzen davon stehen auf Seite 13.

Klassisch: zwei Beobachter

```
namespace ObserverLernprojekt.Klassisch;

public class TemperaturAnzeige : ITemperaturBeobachter
{
    public void Aktualisieren(int temperatur)
    {
        Console.WriteLine($"[Anzeige] {temperatur} °C");
    }
}

public class HitzeAlarm : ITemperaturBeobachter
{
    public void Aktualisieren(int temperatur)
    {
        if (temperatur >= 30)
            Console.WriteLine($"[Alarm] Achtung: {temperatur} °C!");
    }
}
```

Die Anzeige

Hinter dem Doppelpunkt steht `ITemperaturBeobachter`. Damit verspricht `TemperaturAnzeige`, den Vertrag einzuhalten. Bei jeder Benachrichtigung schreibt sie den erhaltenen Messwert in die Konsole.

Der Alarm

`HitzeAlarm` erfüllt denselben Vertrag, tut aber etwas anderes: Nur bei mindestens 30 °C erscheint eine Warnung. Die Wetterstation kennt diese Grenze nicht und muss dafür auch keine `if`-Abfrage enthalten.

Eine stille Reaktion ist trotzdem eine Reaktion

Bei 22 °C wird auch die Methode des Alarms aufgerufen. Die `if`-Bedingung ist dann falsch. Es entsteht keine sichtbare Alarmzeile, obwohl der Beobachter die Meldung erhalten hat.

Wer ruft Aktualisieren auf?

Nicht der Beobachter selbst und auch nicht ein magischer Hintergrundprozess. Der konkrete Aufruf steht in der Station: `b.Aktualisieren(neueTemperatur)`. Er passiert innerhalb von `TemperaturSetzen`. Dieses Zurückrufen einer angemeldeten Reaktion nennt man einen **Callback**.

Die Beobachter fragen den Messwert hier nicht regelmäßig ab. Die Station schickt ihnen die Daten direkt als Parameter. Das ist eine **Push-Variante**. Bei einer Pull-Variante wäre die Benachrichtigung eher „Es hat sich etwas geändert“, und der Beobachter würde die benötigten Daten anschließend von der Quelle lesen.

Die Schreibweise `$" ... {temperatur} ... "` heißt String-Interpolation. C# setzt den Variablenwert in den Text ein. Sie betrifft die Ausgabe, nicht das Observer Pattern.

Klassisch: alles verbinden

Dieser Ausschnitt entspricht dem Kern von Demo.Klassisch. Die Klassen werden erst erzeugt, dann miteinander verbunden. Ohne Anmeldung gibt es keine Benachrichtigung an Anzeige oder Alarm.

```
var station = new K.Wetterstation();  
var anzeige = new K.TemperaturAnzeige();  
var alarm = new K.HitzeAlarm();  
  
station.Anmelden(anzeige);  
station.Anmelden(alarm);  
  
station.TemperaturSetzen(22);  
station.TemperaturSetzen(31);  
  
station.Abmelden(anzeige);  
station.TemperaturSetzen(35);  
station.TemperaturSetzen(35); // Unverändert: keine Meldung.  
  
station.Abmelden(alarm);  
station.TemperaturSetzen(36);
```

K ist in Demo.cs ein Alias für den Namensraum der klassischen Variante. var lässt den Compiler den Variablentyp aus dem Ausdruck rechts ableiten. new erzeugt eine neue Objektinstanz.

Ein Aufruf unter der Lupe: 31 °C

1. Demo ruft TemperaturSetzen(31) auf.
2. Die Station vergleicht 31 mit dem bisherigen Wert 22.
3. Sie speichert 31 und schreibt ihre Stationszeile.
4. Benachrichtigen erstellt eine Kopie der Beobachterliste.
5. Die Anzeige bekommt Aktualisieren(31) und schreibt 31 °C.
6. Der Alarm bekommt Aktualisieren(31) und warnt.
7. Erst danach kehrt TemperaturSetzen zur Demo zurück.

Anmelden ist nicht gleich Benachrichtigen

Unsere Methode Anmelden speichert nur die Referenz. Sie schickt den letzten Messwert nicht automatisch nach. Ein Beobachter, der später dazukommt, erhält erst die nächste Änderung. Soll er sofort den aktuellen Wert sehen, müsste man dieses Verhalten ausdrücklich ergänzen.

Warum Abmelden wichtig ist

Wir entfernen dieselbe Anzeigeinstanz, die wir zuvor angemeldet haben. Eine neu erzeugte Anzeige wäre ein anderes Objekt. Nach der Abmeldung bleibt das bisherige Anzeigeobjekt zwar bestehen, wird aber von dieser Station nicht mehr benachrichtigt.

Events: zuerst den Delegate verstehen

Die Event-Variante verändert die Form der Verbindung: Statt eines Objekts mit einem bestimmten Interface melden wir eine passende **Methode** an. Die Wetterstation muss den Namen der Empfängerklasse weiterhin nicht kennen.

Was ist ein Delegate?

Ein Delegate ist ein typisierter Verweis auf eine aufrufbare Methode. Sein Typ legt fest, welche Parameter und welchen Rückgabewert die Methode haben darf. Ein Delegate kann auch mehrere passende Aufrufe bündeln. Du kannst dir das wie eine Liste passender Rückrufadressen vorstellen.

Für unser Event verwenden wir den vorhandenen Typ `EventHandler<TemperaturEventArgs>`. Dazu passt eine Methode mit dieser Form:

```
void BeiTemperaturAenderung(
    object? sender, TemperaturEventArgs e)
```

sender ist die Quelle des Ereignisses. Unsere Station übergibt sich selbst. **e** ist das Datenpaket mit der Temperatur. Der Empfänger darf sender ignorieren, wenn ihn nur der Wert interessiert. Die Kombination aus Sender und Ereignisdaten folgt dem üblichen .NET-Eventmuster. [2, 4]

Das Datenpaket im Projekt

```
namespace ObserverLernprojekt.MitEvents;

// Ein Datenpaket für alle Empfänger dieser Benachrichtigung.
public class TemperaturEventArgs : EventArgs
{
    public int Temperatur { get; }

    public TemperaturEventArgs(int temperatur)
    {
        Temperatur = temperatur;
    }
}
```

: EventArgs bedeutet hier Vererbung von der .NET-Basisklasse für Ereignisdaten. Der Konstruktor erhält den Messwert und speichert ihn. Der Getter macht die Temperatur lesbar; ohne Setter kann ein Empfänger sie nicht einfach überschreiben.

Drei Dinge auseinanderhalten

Delegate = erlaubte Form der Reaktionsmethode.
 Event = kontrollierte Möglichkeit, solche Reaktionen anzumelden.
 EventArgs = Daten, die beim Auslösen mitgegeben werden.

Das Fragezeichen in `object?` erlaubt einen null-Wert für sender. In unserer konkreten Auslösung wird aber immer die Station als Sender übergeben. `EventHandler<...>` ist ein generischer Delegate-Typ: Zwischen den spitzen Klammern steht der Typ unserer Ereignisdaten.

Events: die Wetterstation

```
namespace ObserverLernprojekt.MitEvents;

public class Wetterstation
{
    public int? Temperatur { get; private set; }

    public event EventHandler<TemperaturEventArgs>? TemperaturGeaendert;

    public void TemperaturSetzen(int neueTemperatur)
    {
        if (Temperatur == neueTemperatur)
            return;

        Temperatur = neueTemperatur;
        Console.WriteLine($"[Station] Neu: {Temperatur} °C");

        // this = Sender; das zweite Argument enthält die Messdaten.
        // ?. verhindert einen Aufruf, wenn niemand angemeldet ist.
        TemperaturGeaendert?.Invoke(
            this, new TemperaturEventArgs(neueTemperatur));
    }
}
```

Was ersetzt die eigene Beobachterliste?

Die Deklaration `public event EventHandler<TemperaturEventArgs>? TemperaturGeaendert;` ist ein feldähnliches Event. C# stellt dafür die Registrierung von Händlern bereit. Unsere Klasse braucht deshalb weder eine eigene Liste noch öffentliche Anmelden- und Abmelden-Methoden.

Was macht ?.Invoke?

Invoke ruft die angemeldeten Methoden auf. ?. führt den Aufruf nur aus, wenn eine Delegate-Instanz vorhanden ist. Ohne angemeldete Handler ist dieses feldähnliche Event null; dann passiert bei dieser Zeile nichts. Das Fragezeichen hinter dem EventHandler-Typ macht diese Möglichkeit im Typ sichtbar.

this ist die aktuelle Station. `new TemperaturEventArgs(neueTemperatur)` erzeugt das Datenpaket. Alle Handler dieser Auslösung erhalten dieses Paket. Zuvor wird der Stationszustand aktualisiert, genau wie in der klassischen Variante.

Warum steht dort event statt nur ein Delegate-Feld?

Das Schlüsselwort begrenzt den Zugriff: Von außen sind bei unserem öffentlichen Event An- und Abmeldung erlaubt. Fremder Code kann es nicht mit einer eigenen Zuweisung ersetzen oder direkt auslösen. Der Aufruf gehört in die deklarierende Klasse. [5]

Kein automatischer Sensor

Das Event erkennt die Änderung nicht von allein. Unser eigener Code vergleicht den Wert, speichert ihn und löst danach das Event aus. Würde die Invoke-Zeile fehlen, bekämen die angemeldeten Handler keine Meldung.

Events: reagieren und abonnieren

```
namespace ObserverLernprojekt.MitEvents;

public class TemperaturAnzeige
{
    public void BeiTemperaturAenderung(
        object? sender, TemperaturEventArgs e)
    {
        Console.WriteLine($"[Anzeige] {e.Temperatur} °C");
    }
}

public class HitzeAlarm
{
    public void BeiTemperaturAenderung(
        object? sender, TemperaturEventArgs e)
    {
        if (e.Temperatur >= 30)
            Console.WriteLine($"[Alarm] Achtung: {e.Temperatur} °C!");
    }
}
```

Diese Klassen brauchen kein gemeinsames Beobachter-Interface. Ihre Methoden müssen zur Signatur des EventHandler-Typs passen: void, Sender und TemperaturEventArgs. Beide Reaktionen sind fachlich dieselben wie vorher.

Die entscheidenden Zeilen aus Demo.MitEvents

```
var station = new E.Wetterstation();
var anzeige = new E.TemperaturAnzeige();
var alarm = new E.HitzeAlarm();

station.TemperaturGeaendert += anzeige.BeiTemperaturAenderung;
station.TemperaturGeaendert += alarm.BeiTemperaturAenderung;

station.TemperaturSetzen(22);
station.TemperaturSetzen(31);

station.TemperaturGeaendert -= anzeige.BeiTemperaturAenderung;
station.TemperaturSetzen(35);
```

`+=` registriert die Reaktionsmethode; `-=` entfernt sie wieder. In der vollständigen Demo folgen wie zuvor derselbe Wert 35, die Abmeldung des Alarms und der Wert 36. [3]

Warum fehlen die Klammern hinter dem Methodennamen?

Mit `anzeige.BeiTemperaturAenderung` übergibst du die Methode für einen späteren Aufruf. Mit Klammern würdest du versuchen, sie sofort aufzurufen. Bei `+=` wird die Methode angemeldet, noch nicht ausgeführt.

Für eine Abmeldung verwenden wir dieselbe Kombination aus Zielobjekt und Methode. Ein neuer `TemperaturAnzeige()`-Aufruf erzeugt einen anderen Empfänger und würde die ursprüngliche Anmeldung nicht entfernen.

Was du in der Konsole siehst

Beide Demos erzeugen denselben fachlichen Ablauf; nur die Überschrift unterscheidet sich. Die folgende **erwartete Ausgabe** ist aus dem Programmcode abgeleitet, kein hier ausgeführtes Konsolenprotokoll.

```
=== Klassisches Observer Pattern ===  
[Station] Neu: 22 °C  
[Anzeige] 22 °C  
[Station] Neu: 31 °C  
[Anzeige] 31 °C  
[Alarm] Achtung: 31 °C!  
--- Anzeige wird abgemeldet ---  
[Station] Neu: 35 °C  
[Alarm] Achtung: 35 °C!  
--- Gleicher Wert: keine Meldung ---  
--- Auch Alarm wird abgemeldet ---  
[Station] Neu: 36 °C
```

Schritt	Wer bekommt einen Callback?	Sichtbare Reaktion
22 °C	Anzeige und Alarm	Nur Anzeige; Alarmbedingung falsch.
31 °C	Anzeige und Alarm	Anzeige und Alarm.
Anzeige abmelden	Niemand	Nur der Demo-Hinweis.
35 °C	Nur Alarm	Nur Alarm.
35 °C erneut	Niemand	Kein Stations- oder Empfängertext.
Alarm abmelden	Niemand	Nur der Demo-Hinweis.
36 °C	Niemand	Nur die eigene Stationszeile.

So kannst du den Ablauf sichtbar nachverfolgen

Setze im Debugger einen Haltepunkt in Aktualisieren beziehungsweise BeiTemperaturAenderung. Starte die passende Demo. Bei 22 °C erreichst du auch den Alarm-Handler, obwohl er keinen Text schreibt. Nach der Abmeldung wird die Anzeige nicht mehr aufgerufen.

Schau dir die Aufrufliste an: Die Benachrichtigung geht von Demo über TemperaturSetzen zum Beobachter. Dadurch siehst du, dass der Beobachter aufgerufen wird und nicht in einer eigenen Schleife auf neue Werte wartet.

Beide Varianten direkt verglichen

Aufgabe	Klassisch	Mit C#-Events
Gemeinsame Form	ITemperaturBeobachter mit Aktualisieren(int).	EventHandler<TemperaturEventArgs> mit passender Methode.
Empfänger speichern	Eigene List<ITemperaturBeobachter>.	Delegate-Aufrufliste des feldähnlichen Events.
Anmelden	station.Anmelden(anzeige);	Event += Handler;
Abmelden	station.Abmelden(anzeige);	Event -= Handler;
Benachrichtigen	foreach und Aktualisieren(wert).	Event?.Invoke(this, daten);
Daten übertragen	int direkt als Parameter.	TemperaturEventArgs als Datenpaket.
Neue Reaktion	Weitere Klasse implementiert das Interface.	Passende Methode wird angemeldet.
Keine Empfänger	Leere Schleife.	?. überspringt den Aufruf.
Doppelte Anmeldung	In unserem Code mit Contains verhindert.	Zweimal += meldet denselben Handler zweimal an.

Was ist gleich geblieben?

Die Station entscheidet, wann sie eine Änderung meldet. Die Empfänger entscheiden, was sie damit tun. Beide Varianten unterstützen mehrere Abonnenten und eine spätere Abmeldung. Die Messwerte, die Alarmgrenze und das Unterdrücken gleicher Folgemessungen sind identisch.

Welche Variante sollst du erklären können?

Für das klassische Pattern solltest du die Rollen, die Beobachterliste und den Methodenaufruf erklären können. Bei Events solltest du zeigen können, wo deklariert, angemeldet, ausgelöst und abgemeldet wird. In der Demo sind diese vier Stellen bewusst getrennt sichtbar.

Die manuelle Variante lässt eigene Registrierungsregeln leicht erkennen. Die Event-Variante verwendet die übliche C#-Schreibweise für Objektbenachrichtigungen. Events machen das Entwurfsmuster also nicht überflüssig: Sie helfen dabei, die gleiche Beziehung auszudrücken.

Keine feste Vorschrift für jede Einzelheit

Die Grenze 30 °C, das Unterdrücken gleicher Werte und der Schutz vor doppelten klassischen Anmeldungen sind Entscheidungen dieses Projekts. Sie gehören nicht zwingend zur Definition des Observer Patterns.

Stolperfallen, die du kennen solltest

1. Events laufen nicht automatisch parallel

In diesem Projekt werden die Reaktionen direkt nacheinander auf dem aufrufenden Thread ausgeführt. Ein langsamer Handler verzögert daher TemperaturSetzen. Das Wort „Event“ bedeutet weder Hintergrundthread noch async. Auch eine besondere zeitliche Ausführungsplanung entsteht nicht automatisch. [2]

2. Ein Fehler kann spätere Empfänger verhindern

Wir fangen Fehler in den Beobachtern nicht ab. Wirft ein früher Beobachter eine Exception, wird der restliche normale Aufrufablauf unterbrochen. Das gilt hier sowohl für die foreach-Schleife als auch für die Event-Auslösung. Eine produktive Anwendung müsste bewusst entscheiden, wie sie solche Fehler behandelt.

3. Vergessene Abmeldungen können Objekte am Leben halten

Die klassische Liste hält Referenzen auf die Beobachter. Bei normalen Event-Anmeldungen hält der Delegate die Ziellinstanz einer Instanzmethode fest. Lebt die Quelle lange, kann dadurch auch ein nicht mehr benötigter Empfänger erreichbar bleiben. Melde ihn ab, wenn er nicht mehr zuhören soll. Das ist besonders bei kurzlebigen Ansichten an langlebigen Diensten relevant. [3]

4. Doppelt anmelden bedeutet bei Events doppelt reagieren

Bei unserem feldähnlichen Event führt zweimal += mit demselben Handler zu zwei Aufrufen. Ein -= entfernt eine passende Registrierung. Unsere klassische Variante verhindert doppelte Einträge vorher selbst. Dieser Unterschied ist im Projekt absichtlich sichtbar.

5. Eine Listenkopie löst nicht alle Probleme

ToArray hält die aktuelle Empfängermenge für eine Benachrichtigung fest. Meldet ein Beobachter während seines Aufrufs einen anderen ab, kann dieser in der laufenden Runde noch aufgerufen werden, weil er bereits in der Kopie steht. Für spätere Meldungen ist er entfernt. Auch eine laufende Delegate-Aufrufliste wird durch Abmeldung nicht rückwirkend umgeschrieben.

Das Projekt ist für einen einzelnen Thread gedacht. Die Kopie schützt nicht umfassend vor gleichzeitigen Zugriffen aus mehreren Threads. Ebenso könnte ein Beobachter durch einen erneuten Aufruf von TemperaturSetzen verschachtelte Benachrichtigungen auslösen. Solche Fälle behandeln wir hier bewusst nicht.

6. Lambdas beim Abmelden wiederfinden

Eine anonyme Funktion kann ein Event-Handler sein. Wenn du sie später abmelden willst, speichere den Delegate zuerst in einer Variablen. Eine neu geschriebene, gleich aussehende Lambda-Funktion ist keine verlässliche Referenz auf die alte Anmeldung. Für den Einstieg sind benannte Methoden wie in unserem Projekt übersichtlicher. [3]

Übungen: erst denken, dann ändern

Starte mit den Vorhersagen. Öffne die Lösungen auf der nächsten Seite erst, nachdem du eine eigene Antwort formuliert hast. Für die Codeaufgaben genügt das enthaltene Projekt.

A. Den Ablauf vorhersagen

1. Ändere in beiden Demos die Messung 31 auf 29. Welche Ausgabe verschwindet? Wird der Alarm bei 29 trotzdem aufgerufen?
2. Kommentiere nur die Abmeldung der Anzeige aus. Welche zusätzlichen Zeilen erwartest du bei 35 und 36 °C? Was passiert beim zweiten Setzen von 35?
3. Melde den Alarm zweimal an. Vergleiche Anmelden(alarm) mit zweimal += alarm.BeiTemperaturAenderung. Was passiert nach einer einzigen Abmeldung?

B. Die Verbindung verstehen

4. Erzeuge eine Anzeige, melde sie aber nicht an. Bekommt sie die Messung? Begründe deine Antwort anhand einer konkreten Codezeile.
5. Setze zuerst 22 °C und melde erst danach die Anzeige an. Zeigt sie sofort 22 an? Was passiert, wenn du erneut 22 setzt und anschließend 23?
6. Erkläre einem Mitschüler ohne den Code zu zeigen: Was ist der Unterschied zwischen einem Event, einem Event-Handler und EventArgs?

C. Eine neue Reaktion ergänzen

7. Ergänze einen FrostAlarm in der klassischen Variante. Er soll ITemperaturBeobachter implementieren und bei Werten unter 0 eine Warnung schreiben. Erzeuge ihn in Demo.Klassisch, melde ihn an und setze testweise -3 °C. Die Wetterstation soll dafür unverändert bleiben.
8. Baue denselben FrostAlarm für die Event-Variante. Verwende BeiTemperaturAenderung(object? sender, TemperaturEventArgs e) und melde die Methode mit += an. Melde sie anschließend mit -= wieder ab.

D. Transferaufgabe

9. Übertrage die Rollen auf einen Online-Shop: Ein Bestellobjekt meldet „Status geändert“. Eine Anzeige und ein Protokoll reagieren. Benenne die Quelle, die Beobachter und geeignete Ereignisdaten. Überlege, ob die Quelle wissen muss, wie das Protokoll seine Daten speichert.

Selbstkontrolle

Du hast den Kern verstanden, wenn du eine neue Reaktion ergänzen kannst, ohne in Wetterstation einen Spezialaufruf für diese Klasse hinzuzufügen. Das Verbinden in der Demo musst du trotzdem ergänzen.

Lösungen und Begründungen

1. Die Alarmzeile für 31 °C entfällt; Station und Anzeige schreiben stattdessen 29 °C. Der Alarm erhält auch 29, aber seine Bedingung ist falsch. Bei der späteren Messung 35 warnt er weiterhin.
2. Zusätzlich erscheint [Anzeige] 35 °C und später [Anzeige] 36 °C. Beim zweiten Setzen von 35 gibt es weiterhin keine Benachrichtigung, weil die Station unveränderte Werte früh abfängt.
3. Klassisch bleibt es bei einer Registrierung. Bei Events reagiert der Alarm zweimal auf einen hohen Wert. Nach einem -= bleibt bei doppelter Registrierung noch ein Eintrag übrig; auch bei 36 °C würde der Alarm dann noch reagieren.
4. Nein. new erzeugt nur das Objekt. Ohne Anmelden landet es nicht in der Liste; ohne += landet seine Methode nicht in der Event-Aufrufliste.
5. Die Anmeldung sendet keinen Wert nach. Ein weiteres Setzen von 22 wird als unverändert unterdrückt. Erst 23 löst eine Meldung aus. Ein sofortiger Anfangswert müsste zusätzlich programmiert werden.
6. Das Event bietet die abonnierbare Benachrichtigung an. Ein Event-Handler ist die angemeldete Reaktionsmethode. EventArgs enthält die übermittelten Ereignisdaten.

7. Klassischer Frostalarm

```
// In einer Datei im Namespace ObserverLernprojekt.Klassisch:
public class FrostAlarm : ITemperaturBeobachter
{
    public void Aktualisieren(int temperatur)
    {
        if (temperatur < 0)
            Console.WriteLine("[Frost] Achtung, unter 0 °C!");
    }
}

// In Demo.Klassisch, nachdem station erzeugt wurde:
var frost = new K.FrostAlarm();
station.Anmelden(frost);
station.TemperaturSetzen(-3);
station.Abmelden(frost);
```

8. Event-Frostalarm

Lege die Klasse im Namespace ObserverLernprojekt.MitEvents an. Entferne das Beobachter-Interface, ersetze die Methode durch die bekannte Event-Handler-Signatur und verwende e.Temperatur < 0. Die Verkabelung lautet:

```
var frost = new E.FrostAlarm();
station.TemperaturGeaendert += frost.BeiTemperaturAenderung;
station.TemperaturSetzen(-3);
station.TemperaturGeaendert -= frost.BeiTemperaturAenderung;
```

9. Quelle: Bestellung. Beobachter: Statusanzeige und Protokoll. Daten: zum Beispiel Bestellnummer, alter und neuer Status. Die Bestellung meldet die Änderung, während das Protokoll selbst über seine Speicherung entscheidet. Der Statuswechsel führt nicht automatisch zu einem Versand echter Nachrichten; das wäre zusätzliche Reaktionslogik.

Spickzettel für den Unterricht

So könntest du das Pattern erklären

„Das Observer Pattern beschreibt eine Eins-zu-viele-Beziehung: Eine Quelle informiert angemeldete Beobachter über Änderungen. Klassisch verwaltet die Quelle eine Liste von Beobachtern mit gemeinsamem Interface. In C# kann man dieselbe Idee mit Events umsetzen: Handler werden mit += angemeldet, vom Sender ausgelöst und mit -= abgemeldet.“

Frage	Kurze Antwort
Wer weiß, dass sich etwas geändert hat?	Die Wetterstation; TemperaturSetzen erkennt es.
Wer entscheidet über die Reaktion?	Der jeweilige Beobachter beziehungsweise Handler.
Müssen Beobachter selbst ständig nachfragen?	In unserem Push-Beispiel nicht.
Ist ein Event ein eigener Thread?	Nein. Unsere Aufrufe sind synchron.
Was bringt ein neuer Beobachter?	Neue Reaktion ohne Änderung der Station.
Was ist mit IObserver<T>?	Ein vorhandenes .NET-Interface; nicht unser eigenes ITemperaturBeobachter.

Einordnung: die weiteren .NET-Interfaces

.NET bietet auch IObservable<T> und IObserver<T>. Dort gehören Subscribe sowie OnNext, OnError und OnCompleted zum Vertrag; eine Anmeldung liefert ein IDisposable zum Beenden des Abonnements. Das ist eine weitere standardisierte API für das Pattern. Für den Vergleich deines Lehrers genügen unsere eigenen klassischen Interfaces und die Event-Variante. [6]

Quellen zum Nachlesen

Die Beispiele und ihre Erklärungen wurden für dieses Lernprojekt erstellt. Die folgenden Microsoft-Dokumentationen belegen die C#- und .NET-Sprachkonzepte. Links geprüft am 11.09.2026.

[1] [C#-Versionsgeschichte: Events bereits in C# 1.0](#)

[2] [Events: Sender, Empfänger und synchrone Aufrufe](#)

[3] [Events abonnieren und abbestellen](#)

[4] [Ereignisse behandeln und auslösen: Delegates und Ereignisdaten](#)

[5] [Das Schlüsselwort event und seine Zugriffsregeln](#)

[6] [Das Observer-Entwurfsmuster in .NET](#)

Zum Projekt gehören README.md, kommentierter Quellcode und Erwartete-Ausgabe.txt. Die Ausgabe ist eine Sollbeschreibung. Ein lokaler Build und Testlauf bleiben erforderlich, weil in der Erstellungsumgebung kein .NET-SDK installiert war.